

ORF307

Introduction &

Course Information

Ioannis Akrotirianakis

Today's topics

- A general overview of the course
- Logistics – Course Policies
- Some applications of Optimization
- Historical timeline of optimization algorithms
- Introduction to Linear Algebra (foundation of Optimization algorithms)

What is Optimization

- Mathematical Optimization is a branch of Applied Mathematics and Operations Research
- It focuses on finding the **best solution** to a problem subject to a set of **constraints**
- **Best solution** → **Best decision** that satisfies the **constraints**

Main components of Optimization problems (*Decision problems*)

- **Objective function(s):**



- It is the **goal** you want to achieve
- Must be expressed in some mathematical form – multivariate function(s)
- **Examples:**
 - Maximize the **returns** on your portfolio
 - Minimize the **transportation cost** of a strawberry farm

Main components of Optimization problems

- **Decision variables:**

- The elements you can change in order to increase (or decrease) your objective
- They represent the **choices** you can make



- **Examples:**

- Which **stocks** you should buy
- Which **route** you should follow to bring your strawberries to your customers

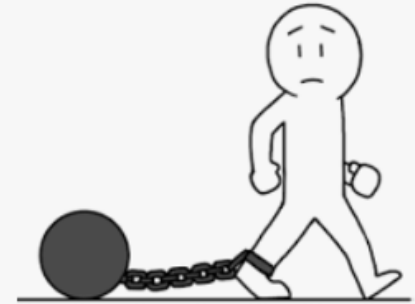
Main components of Optimization problems

- **Constraints:**

- They are the **limitations, rules or the restrictions you must follow**
- They define the **valid choices** you can make

- **Examples:**

- Your **investment capital** is \$10MM, and cannot **allocate** more than 20% of it in one stock
- Your transportation vehicle has **cargo capacity** 3 tons



-
- **Some examples**

Finance (Portfolio Optimization)

- **Decision Variables**

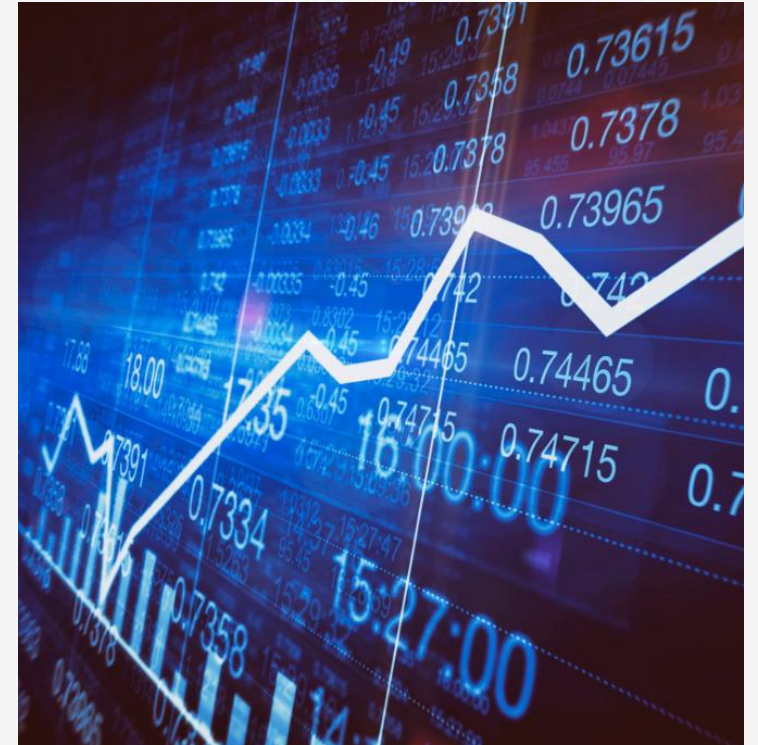
- \$\$\$ to be invested in each asset

- **Constraints**

- Total capital, investment per asset, minimum return

- **Objective**

- Maximize profit or Minimize risk



Optimal control

- **Decision Variables**

- Inputs: thrust, flaps, etc.

- **Constraints**

- System limitations, obstacles, etc.

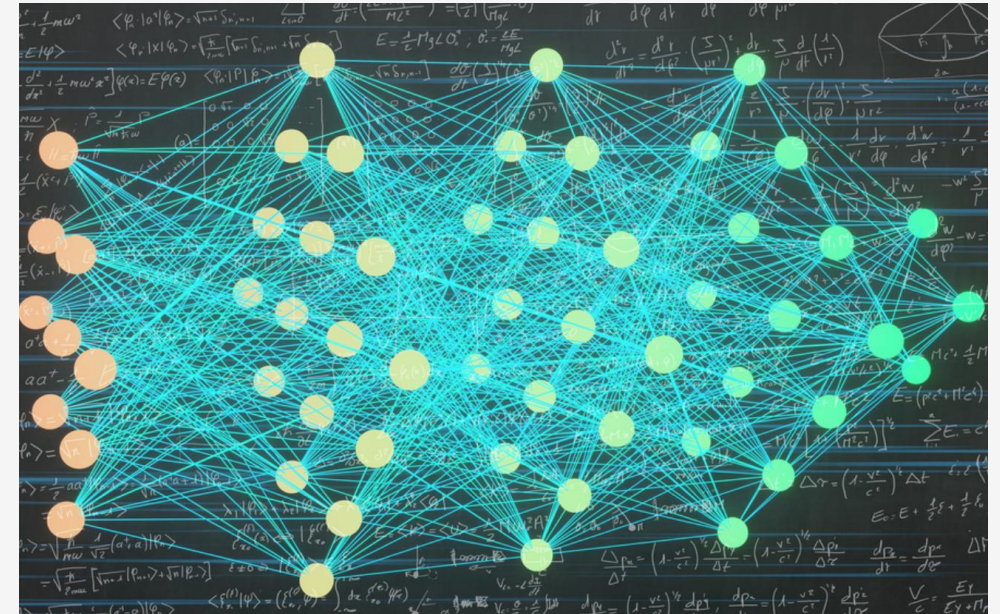
- **Objective**

- Minimize distance to target & fuels used



Machine Learning

- **Decision Variables**
 - Model parameters
- **Constraints**
 - Prior information, parameter limits
- **Objective**
 - Minimize prediction error



- **Course Information**

Meet your teaching staff



Ioannis Akrotirianakis
Lecturer

Office Hours:
Fridays 4 - 6pm

Previous life: Senior Scientist at Siemens and SAS
I was a postdoc at Princeton many years back



Daniel Deza
PhD candidate

Office Hours:
Fridays 11am - 1pm



Dimitrios Lolas
PhD candidate

Office Hours:
Thursdays 5 - 7pm



Shambhavi Suryanarayanan
PhD candidate

Office Hours:
Wednesdays 5-7pm

All office hours take place in Sherrerd rm. 107 (ORFE library room)

Weekly schedule

Lectures:

Tuesdays and **Thursdays**, 10:40AM – 12:00PM → Lewis Lib 120

For **lecture slides** check Canvas/Modules/

For **supporting Python notebooks** check the course **GitHub Companion:**

<https://github.com/ORF307/companion>

Weekly schedule

Precepts:

P01 → **Tuesdays** 7:30 – 8:20pm → Friend Center 108

P02 → **Tuesdays** 7:30 – 8:20pm → Andlinger 017

P03 → **Wednesdays** 7:30 – 8:20pm → Friend Center 108

Precepts start this week!

For **problems/questions** check Canvas/Modules/

For **supporting Python notebooks** check <https://github.com/ORF307/companion>

Note: There are older versions of the problems/questions in the Companion;

Please use the **latest versions** on Canvas/Modules/

Course material

- **Prerequisites**

- Linear algebra (MAT202 and/or MAT204)
- Basic computer programming knowledge (PYTHON)

- **Python environment for HWs**

- You are free to create your own Python environment and create PDF file of your HW solutions
PDF file → Gradescope submission
- Two popular ways to work with Python:

GitHub Codespaces: <https://github.com/ORF307/companion> or Canvas/Modules/

Docker image: <https://github.com/bstellato/optimization-docker#> or Canvas/Modules/

Course material

- **Readings**

- Highly based on the following books (all digitally available)
 - **Boyd & Vandenberghe**: Introduction to Applied Linear Algebra – Vectors, Matrices, and Least Squares
 - **Bertsimas & Tsitsiklis**: Introduction to Linear Optimization
 - **Vanderbei**: Linear Programming: Foundations & Extensions

Grading

- **Homework: 30%**
 - 8 biweekly PSETs with both theoretical and computational problems
 - Collaboration is encouraged! Extensions will not be given!
Exceptions to the rule → see Syllabus on Canvas
- **Two midterms: 40%**
 - We drop the lowest score when calculating the final grade
 - 80 minutes written exam in class. No collaboration.
- **Final project: 30%**
 - Theoretical and coding component. No collaboration and No AI help.

Generative AI policy!

- The use of Generative AI (GAI) is NOT allowed during any exam (the two Midterms and Final).
- You are NOT allowed to ask GAI tools to provide the solution to the questions in the homework assignments.
- You may use GAI to analyze past homework assignments or exams (i.e., assignments you have already submitted for grading, or past exams we will provide you for practice).
- You may also use GAI tools to fix Python syntax errors or other technical/computer issues you may encounter during homework assignments.
- If you use GenAI tools on an assignment or take-home exam, you must *declare it, describe how you used the tool, and include both the prompt and the relevant output*. Using these tools without disclosing when and how you used them is a violation of the University's [academic regulations](#)[Links to an external site.](#) (see **Section 2.4.6.**), and can have serious consequences.

Software (all open-source)

- **All computational exercises will be done in Python**

- source code or Jupyter notebooks are both OK

- **Numerical computation packages**

- Numpy and scipy

- **Optimization solver**

- CvxPy (<https://www.cvxpy.org/>)

$$\begin{array}{ll} \text{minimize} & c^T x \\ \text{subject to} & Ax \leq b \end{array}$$

```
x = cp.Variable(n)
prob = cp.Problem(
    cp.Minimize(c.T@x),
    [A @ x <= b]
)
prob.solve()
print("The optimal value is", prob.value)
print("The solution x is", x.value)
```

Learning goals

- **Formulate** various **decision-making problems** from a wide variety of disciplines as Least-Squares, Linear and Integer Optimization problems
 - **Apply** the most suitable **optimization tools** to solve decision-making problems
 - **Understand** algorithmic and problem **complexity**
 - Which algorithms are slower, and which are faster
 - Which problems are easier, and which are harder
- Select the **right algorithm** and the **right model**

Complexity of Optimization problems

- In general, they are very hard to solve!
- Some of them are intractable (MIP)
- They require prohibitively long computational times and resources
- Approximate solutions are acceptable

- Computational efficient algorithms exist for some special classes:
- Least squares problems
- Linear Optimization problems
- Convex Optimization problems

What we will cover in this course

- **Least-squares problems**

- Solving linear systems in practice
- Modelling and applications
- Multi-objective least squares
- Constrained least squares

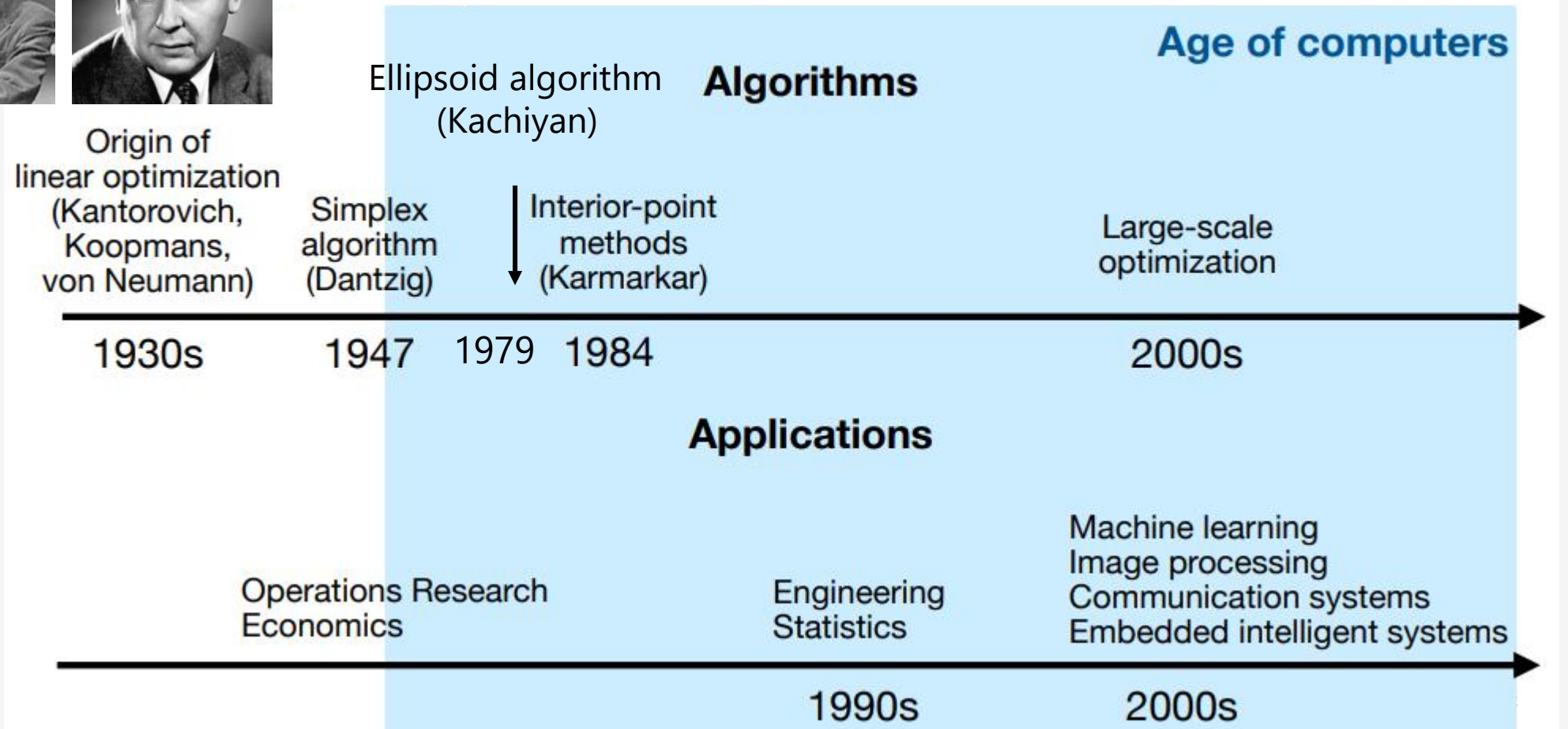
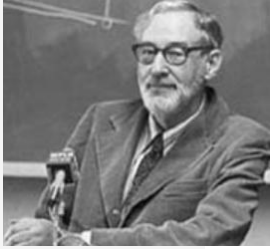
- **Linear Optimization:**

- Modeling and applications
- Geometry
- The Simplex method
- Duality
- Network optimization
- Interior Point Methods

- **Extensions**

- Mixed-Integer Optimization (MIP)
- Branch-and-Bound algorithms

Brief History of Optimization



-
- **Notation and basic definitions**

Vectors

- An **n -dimensional** vector is denoted by: $x = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}, \quad x \in R^n$
- The **transpose** of a vector x is defined as: $x = (x_1, x_2, \dots, x_n)^T$
- **Special vectors:**
 - **Zero vector:** $x = 0$ ($x_i = 0, \forall i$) $\rightarrow 0 = (0, 0, \dots, 0)^T$
 - **All ones vector:** $x = \mathbf{1}$ ($x_i = 1, \forall i$) $\rightarrow \mathbf{1} = (1, 1, \dots, 1)^T$
 - **Unit vector:** $x = e_i$, where $e_i = (0, 0, \dots, 0, 1, 0, \dots, 0)^T$ Only the i -th element is 1 and all others are 0

Vector operations

Addition: $\mathbf{x} + \mathbf{y} = \begin{pmatrix} x_1 + y_1 \\ x_2 + y_2 \\ \vdots \\ x_n + y_n \end{pmatrix}, \quad \mathbf{x}, \mathbf{y} \in \mathbb{R}^n$

Scalar multiplication: $a\mathbf{x} = \begin{pmatrix} ax_1 \\ ax_2 \\ \vdots \\ ax_n \end{pmatrix}, \quad a \in \mathbb{R}$

Inner (dot) product: $\mathbf{x}^T \mathbf{y} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}^T \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = (x_1, x_2, \dots, x_n) \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \sum_{i=1}^n x_i y_i = \sum_{i=1}^n y_i x_i = \mathbf{y}^T \mathbf{x}$

Vector inner (dot) product examples

- **Special cases**

- $e_i^T x = x_i$ (pick the i -th entry)
- $1^T x = \sum_{i=1}^n x_i$ (sum of all entries)
- $\frac{1}{n} 1^T x = \frac{1}{n} \sum_{i=1}^n x_i$ (average of vector elements)
- $x^T x = \sum_{i=1}^n x_i^2$ (sum of squares of all entries)

Vector inner (dot) product examples

Examples of vectors

- $c \in R^3$ with entries $c = (0,0,0)^T$ may be the **origin** of R^3
- $c \in R^3$ may represent the **black** color. The **green** color is $g = (0,1,0)$.
- Vectors hold the values of **time-series** (e.g., the daily temperatures in Princeton for 10 years)
- Your **portfolio** of 10 stocks is defined as a 10-dimensional vector $w \in R^{10}$

Example 1:

p : vector of prices
 q : vector of quantities

Total cost: $p^T q$

Example 2:

s : vector of holding assets (shares)
 p : vector of share prices

Total portfolio value: $s^T p$

Example 3:

t : vector of temperatures
 n : number of temperatures recorded

Average temperature: $\frac{1}{n} 1^T t$

More inner product examples

- **Portfolio returns**

- Vector of **fractional allocations** of your capital $\rightarrow w \in R^n$
 - w_i : percentage of your capital allocated to company i
- Vector of **fractional returns** $\rightarrow r \in R^n$

$$r_i = \frac{p_i^{final} - p_i^{initial}}{p_i^{initial}}, \quad i = 1, 2, \dots, n$$

- **Fractional return of your portfolio** $\rightarrow r^T w$

Vector norms

Vector norms are functions of the form $||\cdot||: R^n \rightarrow R$ with special properties

Properties:

- For all $x, y \in R^n$ and $a \in R$
- **Homogeneous:** $||ax|| = |a| ||x||$
- **Triangle inequality:** $|x + y| \leq |x| + |y|$
- **Nonnegativity:** $||x|| \geq 0$
- **Definiteness:** $||x|| = 0$ iff $x = 0$

Vector norms

Vector norms are functions of the form $||\cdot||: R^n \rightarrow R$ with special properties

Properties:

- For all $x, y \in R^n$ and $a \in R$
- **Homogeneous:** $||ax|| = |a| ||x||$
- **Triangle inequality:** $|x + y| \leq |x| + |y|$
- **Nonnegativity:** $||x|| \geq 0$
- **Definiteness:** $||x|| = 0$ iff $x = 0$

ℓ_2 (Euclidean) norm:

$$||x||_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2} = \sqrt{x^T x}$$

ℓ_1 (Manhattan) norm:

$$||x||_1 = |x_1| + |x_2| + \cdots + |x_n|$$

ℓ_∞ (Infinity) norm:

$$||x||_\infty = \max\{|x_1|, |x_2|, \dots, |x_n|\} = \max_{i=1 \dots n} \{|x_i|\}$$

Angle between vectors (geometrical view)

- The angle θ between two vectors x, y satisfies

$$x^T y = \|x\| \|y\| \cos(\theta)$$

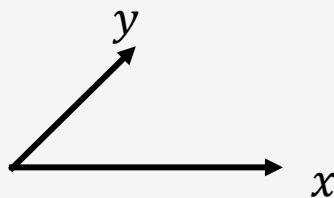
Interpretation of $x^T y$:

the **length of x** multiplied by
the **length of the projection of y onto x**

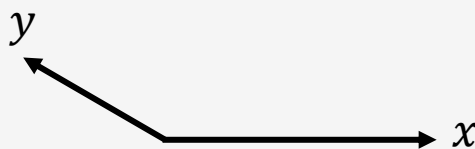
- Orthogonal vectors** ($\theta = \pi/2$) $\rightarrow x^T y = \mathbf{0}$



- If $\theta < \pi/2$ then $x^T y > 0$



- If $\theta > \pi/2$ then $x^T y < 0$



Cauchy-Schwarz inequality

- Very important property in Optimization and Linear Algebra!

$$|x^T y| \leq \|x\| \|y\|$$

(relationship between the inner product of two vectors and their lengths)

Properties:

- It holds for all vectors with the same dimension only
- $|x^T y| = \|x\| \|y\|$ iff x, y are aligned (parallel) → WHY?

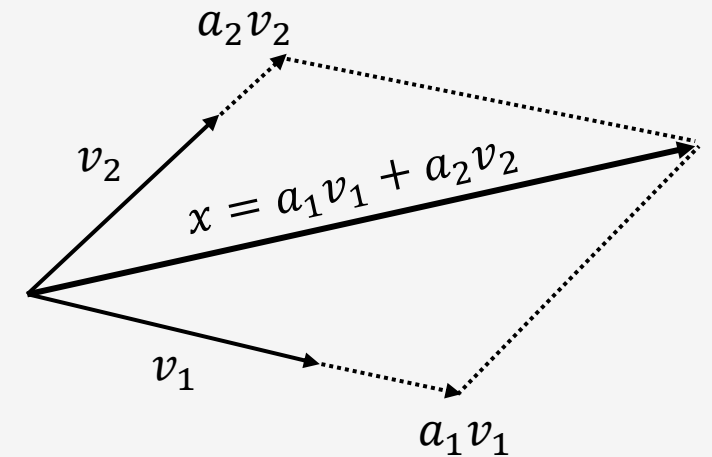
Linear independence of vectors

- **Definition:** A nonempty set of vectors $V = \{v_1, v_2, \dots, v_n\}$ is **linear independent** if the only solution to the equation

$$a_1 v_1 + a_2 v_2 + \dots + a_n v_n = 0$$

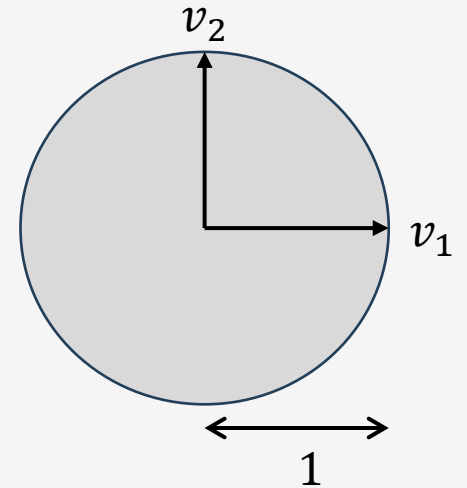
is the trivial one (i.e., $a_1 = a_2 = \dots = a_n = 0$)

- **Properties:**
 - The set $V = \{v_1, v_2, \dots, v_n\}$ forms a **basis** of R^n
 - **None** of the vectors v_i can be written as **linear combination** of the other vectors in the **basis**.
 - **Every vector** in R^n can be written as a **linear combination** of the **basis vectors**.



Orthonormal vectors

- **Definition:** The vectors $v_1, v_2, \dots, v_k \in R^n$ are called **orthonormal** iff
 - $v_i^T v_j = 0$ for all $i \neq j$
 - $\|v_i\|=1$ for all $i = 1, 2, \dots, k$
- **Definition:** The set of vectors $V = \{v_1, v_2, \dots, v_n\}$ forms an **orthonormal basis** of R^n iff $v_1, v_2, \dots, v_n$ are linear independent and orthonormal vectors



Flop counts

- Flops → **F**loating-point **o**peration
- Computers store real numbers in floating-point format
- Basic arithmetic operations (+, −, ×, ÷) are called floating-point operations
- **Algorithm complexity** = **total computational cost** = **total number of flops needed as a function of dimensions** (of vectors, metrics, etc.)
- Modern computers can perform: **1 Gflops/sec** (10^9 flops/sec)

Complexity of vector operations

- **Examples**

- $x + y$ needs n **additions**; hence: n flops
- $x^T y$ needs n **multiplications** and $n - 1$ **additions**; hence $2n - 1$ flops
 - In complexity theory **coefficients** of n can be omitted
 - The **leading term** without the coefficient is considered only
 - Most vector operations have complexity n

Next lecture

- Matrix operations and solving linear systems in practice
 - Matrix operations on computers
 - Solving linear systems
 - Matrix factorization
- Do not forget to check this lecture's Python notebooks in the Companion
